

CSA0609 - DESIGN AND ANALYSIS OF ALGORITHM

Binary Search with Big-O Analysis

1. Aim

To write a Python program to search for an element in a **sorted array** using the **Binary Search algorithm** and analyze its Big-O complexity.

2. Algorithm

1. Start with a sorted array.
2. Set $low = 0$ and $high = n - 1$.
3. Find the middle element.
4. If the middle element equals the search element, return its position.
5. If the search element is greater, search the right half.
6. Otherwise, search the left half.
7. Repeat until the element is found or the search range becomes empty.

3. Pseudocode

Algorithm BinarySearch(A, key)

Input: Sorted array A and search element key

Output: Position of key

1. $low \leftarrow 0$
2. $high \leftarrow \text{length}(A) - 1$
3. While $low \leq high$ do
4. $mid \leftarrow (low + high) / 2$
5. If $A[mid] = key$ then
6. Return mid
7. Else If $A[mid] < key$ then
8. $low \leftarrow mid + 1$
9. Else

10. $\text{high} \leftarrow \text{mid} - 1$

11. End If

12. End While

13. Return -1

End Algorithm

4. Python Code

```
def binarySearch(A, key):
```

```
    low = 0
```

```
    high = len(A) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if A[mid] == key:
```

```
            return mid
```

```
        elif A[mid] < key:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1
```

```
# Sorted array
```

```
A = [10, 20, 30, 40, 50, 60, 70]
```

```
key = 50
```

```
result = binarySearch(A, key)
```

```
if result != -1:
```

```
    print("Element found at index:", result)
```

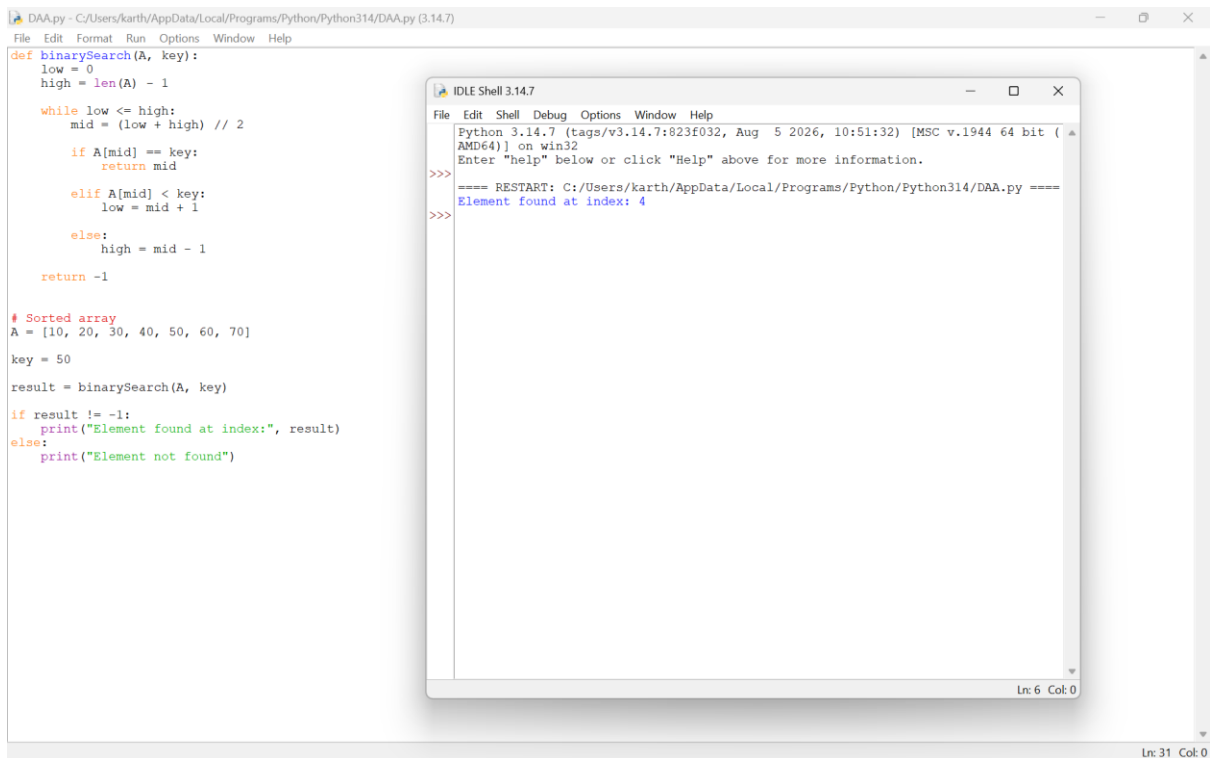
```
else:
```

```
    print("Element not found")
```

5. Output

Element found at index: 4

6. Output Screenshot



The screenshot shows a Python IDE window titled 'DAA.py - C:/Users/karth/AppData/Local/Programs/Python/Python314/DAA.py (3.14.7)'. The code defines a `binarySearch` function that takes a list `A` and a `key`. It uses a `while` loop to find the `key` in the sorted array `A`. The array `A` is `[10, 20, 30, 40, 50, 60, 70]` and the `key` is `50`. The function returns the index of the `key` if found, otherwise `-1`. The output window shows the result: `Element found at index: 4`.

```
def binarySearch(A, key):
    low = 0
    high = len(A) - 1

    while low <= high:
        mid = (low + high) // 2
        if A[mid] == key:
            return mid
        elif A[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Sorted array
A = [10, 20, 30, 40, 50, 60, 70]
key = 50
result = binarySearch(A, key)
if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/DAA.py ====
>>>
Element found at index: 4
>>>
```

7. Complexity Analysis

Binary Search reduces the search space by **half** after every comparison.

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots$$

Therefore:

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Space Complexity: $O(1)$ for the iterative implementation.

8. Result

Thus, the element was successfully searched using **Binary Search**. The worst-case time complexity is $O(\log n)$ and the space complexity is $O(1)$.